GraphRAG e KAG sobre Neo4j: Recuperação Híbrida e Raciocínio por Forma Lógica para Perguntas Multi-hop com Rastreabilidade

Caio Dantas da Silveira[1]

[1]Curso de Bacharelado em Ciência da Computação
Disciplina Tendências em Ciência da Computação – Prof. Ricardo Roberto de Lima
caiodantas45@icloud.com

Abstract. *Large Language Models hallucinate, struggle with multi-hop reasoning and rarely expose the evidence behind their answers. We present an open-source question answering system that turns unstructured Portuguese texts into a Labeled Property Graph in Neo4j and answers questions through (i) hybrid retrieval that fuses vector, full-text and entity-linking signals with Reciprocal Rank Fusion, (ii) multi-hop Cypher traversal enriched by PageRank and Louvain communities and (iii) a Knowledge-Augmented Generation (KAG) layer that compiles the question into a closed-vocabulary logical form executed as validated Cypher templates. Answers must cite numbered graph facts, false premises trigger abstention and a post-generation verifier computes a confidence score. On a held-out set of 14 questions in the History of Computing domain, run fully offline, KAG reached 79% accuracy versus 14% for a vector-only RAG baseline, with zero unsupported answers.*

Resumo. *Modelos de Linguagem de Grande Porte alucinam, falham em raciocínio multi-hop e raramente mostram a evidência das respostas. Apresentamos um sistema de perguntas e respostas de código aberto que transforma textos em português em um grafo de propriedades rotulado (LPG) no Neo4j e responde perguntas por meio de (i) recuperação híbrida que funde sinais vetoriais, full-text e de entity linking com Reciprocal Rank Fusion, (ii) travessia multi-hop em Cypher enriquecida por PageRank e comunidades de Louvain e (iii) uma camada de Geração Aumentada por Conhecimento (KAG) que compila a pergunta em uma forma lógica de vocabulário fechado, executada como templates Cypher validados. As respostas precisam citar fatos numerados do grafo, premissas falsas levam à abstenção e um verificador pós-geração calcula um nível de confiança. Em um conjunto de teste separado com 14 perguntas sobre História da Computação, executado inteiramente offline, o KAG atingiu 79% de acurácia, contra 14% de um RAG puramente vetorial, sem nenhuma resposta não suportada.*

1 Introdução

A Geração Aumentada por Recuperação (RAG) reduz alucinações ao condicionar o modelo de linguagem a trechos recuperados de uma base documental [Lewis et al., 2020]. Na sua forma mais comum, contudo, a recuperação é feita apenas

por similaridade vetorial entre a pergunta e trechos de texto. Esse desenho tem duas limitações conhecidas: perguntas que exigem combinar fatos dispersos em documentos diferentes (*multi-hop*) raramente são semelhantes a um único trecho [Yang et al., 2018], e a resposta final não expõe o raciocínio que conecta a evidência à conclusão, o que dificulta a auditoria e favorece alucinações [Ji et al., 2023].

Grafos de conhecimento oferecem uma estrutura explícita para esse tipo de composição [Hogan et al., 2021, Pan et al., 2024]. Abordagens recentes de GraphRAG [Edge et al., 2024, Peng et al., 2024] constroem grafos a partir de texto e os usam como contexto, enquanto o KAG [Liang et al., 2024] propõe decompor a pergunta em uma forma lógica executada sobre o grafo.

Este trabalho descreve o projeto final da disciplina Tendências em Ciência da Computação: um sistema completo, de código aberto, que cobre os quatro pilares exigidos — *pipeline* Text-to-Graph, modelagem LPG no Neo4j, recuperação híbrida multi-hop e interface com visualização e rastreabilidade. As contribuições são:

- uma modelagem LPG em duas camadas (lexical e de conhecimento) com proveniência na aresta, constraints de unicidade e os três tipos de índice do Neo4j (RANGE, FULLTEXT e VECTOR);
- um executor KAG em que o LLM apenas escolhe operadores de um vocabulário fechado, enquanto o Cypher é gerado por *templates* validados contra a ontologia, eliminando Cypher alucinado;
- um esquema de mitigação de alucinações em três estágios (antes, durante e depois da geração), com abstenção como resultado de primeira classe;
- uma avaliação comparativa entre RAG vetorial, GraphRAG e KAG em conjuntos de desenvolvimento e de teste separados, totalmente reprodutível sem GPU nem API paga.

2 Fundamentação e Trabalhos Relacionados

Grafos de propriedades rotulados. No modelo LPG, nós e arestas possuem rótulos e pares chave–valor [Angles, 2018]. O Neo4j implementa esse modelo com a linguagem declarativa Cypher [Robinson et al., 2015] e oferece, a partir da versão 5, índices RANGE (sucessores dos B-trees), índices full-text baseados em Lucene e índices vetoriais aproximados baseados em HNSW [Malkov and Yashunin, 2020].

RAG e GraphRAG. Lewis et al. [Lewis et al., 2020] combinaram um recuperador denso a um gerador *seq2seq*. O GraphRAG da Microsoft [Edge et al., 2024] extrai entidades e relações com um LLM, detecta comunidades hierárquicas com Leiden [Traag et al., 2019] e gera resumos por comunidade para perguntas globais. O levantamento de Peng et al. [Peng et al., 2024] organiza a área em indexação, recuperação e geração baseadas em grafos.

KAG. Liang et al. [Liang et al., 2024] propõem alinhar texto e grafo por indexação mútua e resolver perguntas por raciocínio guiado por forma lógica, combinando

recuperação, raciocínio sobre o grafo e cálculo. Nosso executor adota a mesma ideia com um conjunto reduzido de operadores (encadeamento com interseção e caminho mínimo).

Algoritmos de grafo. PageRank [Page et al., 1999] mede a centralidade das entidades; Louvain [Blondel et al., 2008] agrupa entidades densamente conectadas. Para fundir listas ranqueadas heterogêneas usamos *Reciprocal Rank Fusion* (RRF) [Cormack et al., 2009].

3 Sistema Proposto

A Figura 1 resume a arquitetura. Toda a implementação está em Python, com o driver oficial do Neo4j; os componentes que usam LLM aceitam qualquer API compatível com a da OpenAI (incluindo modelos locais via Ollama) e têm alternativa determinística, de modo que o sistema funciona também sem LLM.

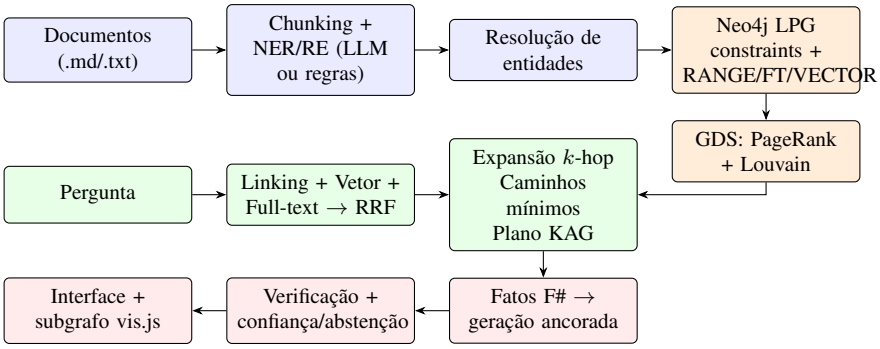


Figura 1. Arquitetura: construção do grafo (azul), armazenamento e algoritmos (laranja), recuperação (verde) e geração com verificação (vermelho).

3.1 Domínio, dados e modelagem LPG

O domínio escolhido é a História da Computação, rico em relações encadeadas (orientador → instituição; criador → linguagem → laboratório). O corpus tem 15 documentos em português, um por personalidade ou tema.

O grafo possui duas camadas ligadas por `MENTIONS`. A camada lexical (`Document`, `Chunk`) guarda o texto e seus *embeddings*; a camada de conhecimento usa um rótulo base `Entity` combinado a um rótulo de tipo (`Person`, `Organization`, `Location`, `Technology`, `Concept`, `Award`). O rótulo base permite um único índice vetorial e um único full-text sobre todas as entidades e uma constraint global de unicidade; o rótulo de tipo mantém as consultas seletivas. A ontologia define 15 tipos de relacionamento com domínio e alcance (ex.: `ADVISED`: Person→Person; `DEVELOPED_AT`: Technology→Organization). Cada aresta de domínio guarda `sources` (ids dos *chunks* que a afirmam), `year`, `confidence` e `mention_count`: quando o mesmo fato aparece em dois documentos, o `MERGE` acumula as fontes em vez de duplicar a aresta.

O esquema é criado com 10 constraints de unicidade (`Entity.id`, uma por rótulo de tipo e as de `Document`, `Chunk` e `Community`), índices RANGE em nome, tipo, PageRank, comunidade, documento e no ano da relação `RECEIVED`, índices full-text so-

bre nomes/aliases e sobre o texto dos *chunks* (analisador `brazilian`) e índices vetoriais de cosseno para *chunks* e entidades:

```
CREATE CONSTRAINT entity_id IF NOT EXISTS FOR (e:Entity) REQUIRE e.
    id IS UNIQUE
CREATE FULLTEXT INDEX entity_fulltext IF NOT EXISTS FOR (e:Entity)
    ON EACH [e.name, e.aliases_text]
CREATE VECTOR INDEX chunk_embedding IF NOT EXISTS FOR (c:Chunk) ON c
    .embedding
  OPTIONS {indexConfig: {`vector.dimensions`: 384, `vector.
      similarity_function`: 'cosine'}}
```

3.2 Pipeline Text-to-Graph

Os documentos são divididos em *chunks* por parágrafo, respeitando limites de sentença. Dois extratores compartilham a mesma interface. O **extrator LLM** recebe a descrição da ontologia e um exemplo, e devolve JSON com entidades e relações; a saída é validada: tipos fora do esquema são descartados (modo estrito) ou registrados (modo dinâmico), relações com direção invertida são corrigidas pela assinatura domínio/alcance e relações cujas entidades não foram declaradas são rejeitadas. O **extrator por regras**, usado no modo *offline*, faz NER por dicionário com casamento do maior trecho — o que distingue, por exemplo, "Prêmio Turing", "Máquina de Turing" e "Turing", e as duas cidades chamadas Cambridge — e RE por padrões lexicais aplicados ao texto entre duas menções, com restrições de tipo. Três heurísticas evitaram os erros mais frequentes: usar apenas a pista mais próxima da segunda menção; ignorar o par quando o trecho após uma entidade intermediária começa com particípio ("a linguagem C foi influenciada pela linguagem B, *criada por* Ken Thompson" não gera *Thompson criou C*); e ignorar adjuntos preposicionais da entidade intermediária ("fundou o W3C *no* MIT" não gera *fundou o MIT*).

A **resolução de entidades** é feita em cascata, sempre restrita ao mesmo tipo: alias normalizado (sem acentos e pontuação), sobrenome único de pessoa, igualdade de tokens significativos, similaridade de Levenshtein acima de 0,90 — com uma guarda que impede fundir nomes que diferem em tokens curtos ou números, como "linguagem B" e "linguagem C" — e, opcionalmente, similaridade de *embeddings*. Os ids são determinísticos (`person:alan_turing`), o que torna a ingestão idempotente e incremental. O grafo resultante tem 99 entidades e 134 relações de domínio.

Após a ingestão, PageRank e Louvain são executados no Neo4j Graph Data Science (com *fallback* em NetworkX quando o plugin não está disponível) e cada comunidade recebe um resumo, gerado por LLM ou por *template*.

3.3 Recuperação híbrida e multi-hop

Dada uma pergunta, o sistema (1) recupera *chunks* por busca vetorial; (2) liga menções da pergunta a entidades do grafo (*entity linking*) e detecta nomes próprios que não existem no grafo; (3) consulta os índices vetorial e full-text de entidades e coleta as entidades mencionadas nos melhores *chunks*; (4) funde essas quatro listas com RRF ($k = 60$), dando prioridade às âncoras ligadas exatamente; (5) expande a vizinhança das

âncoras com `[:REL*1..k]`, ordenando os caminhos por comprimento e PageRank; e (6) calcula `allShortestPaths` entre pares de âncoras. Cada aresta recuperada vira um fato numerado (F1, F2, ...), pontuado por origem (plano KAG $>$ caminho mínimo $>$ expansão), por coincidir com as relações citadas na pergunta e pela centralidade dos nós; os 30 melhores formam o contexto, junto com os *chunks* de proveniência e os resumos das comunidades das âncoras.

3.4 KAG: raciocínio por forma lógica

No modo KAG, a pergunta é compilada em um plano com dois operadores. `chain` é uma sequência de passos `(origem)-[REL]->(variável)`, em que a origem é uma âncora ou uma variável anterior, com direção e tipo-alvo; uma variável atribuída por dois passos é a interseção dos resultados. `path` busca caminhos mínimos entre âncoras, filtrados pelo tipo da resposta e pelas relações da pergunta. Para "Onde trabalhava o orientador de Alan Turing?", o plano é `Alan Turing <-[ADVISED]- x1 -[WORKED_AT]-> x2`.

O plano é produzido por um LLM, que recebe o esquema e as âncoras e responde em JSON validado, ou, sem LLM, por um planejador heurístico que associa pistas lexicais a relações, ordena os passos pela distância à âncora e decide a direção pela assinatura domínio/alcance da ontologia (para relações Pessoa→Pessoa, pela posição da âncora em relação ao verbo). Em ambos os casos o LLM nunca escreve Cypher: cada passo é traduzido em um *template* parametrizado cujos rótulos e tipos interpolados são verificados contra a ontologia. Após a execução, uma poda para trás remove valores intermediários que não levam à resposta, de modo que só os fatos efetivamente usados sejam citados. Complementarmente, quando há LLM e Neo4j, um módulo Text2Cypher gera consultas livres protegidas por validação estática (somente leitura, rótulos e tipos do esquema, `LIMIT` obrigatório), `EXPLAIN` prévio com uma tentativa de autocorreção e execução em transação de leitura.

3.5 Geração ancorada e mitigação de alucinações

A mitigação atua em três estágios. **Antes da geração**, o contexto contém apenas fatos numerados com fonte, caminhos e trechos; se a pergunta menciona uma entidade inexistente no grafo (premissa não verificável, como "Prêmio Nobel" de Dennis Ritchie), o sistema se abstém sem consultar o gerador. **Durante**, o *prompt* exige uso exclusivo do contexto, uma citação por frase e uma frase padrão de abstenção, com temperatura zero; sem LLM, um gerador extrativo verbaliza os fatos do plano KAG. **Depois**, um verificador mede a cobertura de citações válidas por frase, a fração de entidades da resposta presentes no contexto recuperado e a ocorrência de nomes próprios que não existem nem no grafo nem nos trechos. A confiança final é

$$c = 0{,}45\, r + 0{,}30\, q + 0{,}25\, g, \tag{1}$$

em que r é a confiança da recuperação (âncoras ligadas, plano executado, fatos encontrados), q a cobertura de citações e g o aterramento de entidades, com penalidades multiplicativas para citações inválidas ($\times 0{,}7$) e entidades não aterradas ($\times 0{,}5$). Respostas com $c < 0{,}35$ são substituídas por abstenção.

3.6 Interface e rastreabilidade

A interface em Streamlit mostra a resposta com citações e o nível de confiança, e organiza a rastreabilidade em abas: o subgrafo interativo (vis-network), com âncoras, resposta e caminho do raciocínio destacados; a tabela de fatos com origem e *chunks* de procedência; o plano KAG e sua execução passo a passo; todas as consultas Cypher executadas, com parâmetros e tempo; o relatório de verificação com o contexto exato enviado ao gerador; e os sinais de recuperação (âncoras, sementes RRF, *chunks* e comunidades). Há ainda páginas de ingestão de novos documentos, exploração do grafo e avaliação. A Figura 2 mostra o subgrafo de contexto de uma pergunta multi-hop.

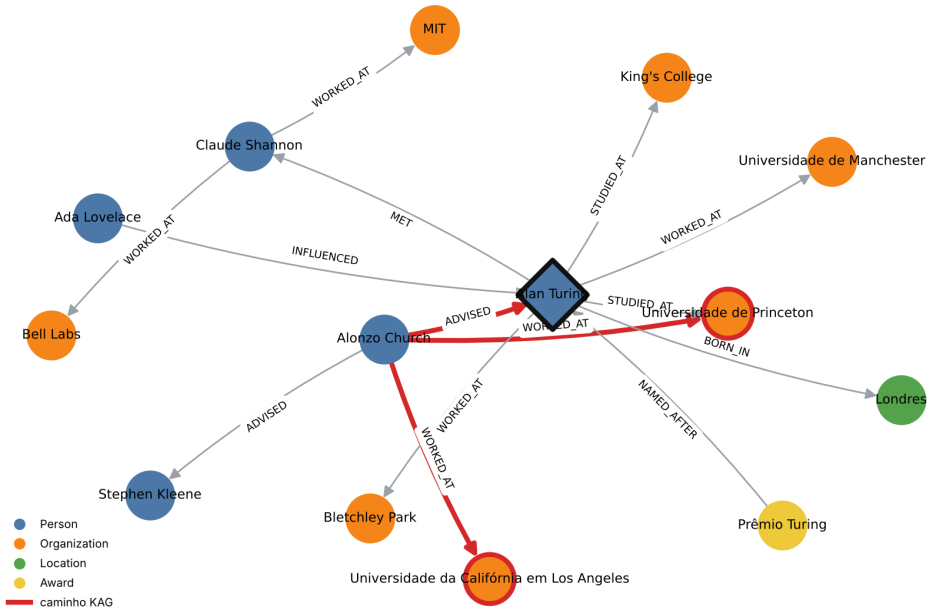


Figura 2. Subgrafo de contexto para "Onde trabalhava o orientador de Alan Turing?". A âncora aparece como losango, as arestas do plano KAG em vermelho e as respostas com borda vermelha.

4 Avaliação Experimental

4.1 Configuração

Comparamos três modos: **RAG vetorial** (somente *chunks*), **GraphRAG** (recuperação híbrida, expansão, caminhos mínimos e comunidades) e **KAG** (GraphRAG + forma lógica). Todos os experimentos foram executados no modo *offline*: extração por regras, *embeddings* por *feature hashing* de palavras e trigramas (384 dimensões) e gerador extrativo, o que os torna determinísticos e reprodutíveis com um único comando.

Usamos dois conjuntos de perguntas. O de **desenvolvimento** (21 perguntas: 7 de um salto, 10 multi-hop, 1 de caminho e 3 sem resposta no corpus) foi usado durante a construção do sistema, inclusive para ajustar o planejador heurístico, e por isso é otimista. O de **teste** (14 perguntas: 5, 6, 1 e 2, respectivamente) foi escrito depois, com outras formulações, e não foi usado para ajustes.

As métricas são: *acurácia* (a resposta contém todas as entidades esperadas, ou o sistema se absteve quando não há resposta); *precisão da resposta* (fração das entidades da linha de resposta que são esperadas); *recall de contexto* (fração das entidades esperadas presentes no contexto entregue ao gerador, independente do gerador); *abstenção correta*; e *respostas não suportadas*, *proxy* de alucinação que conta respostas dadas a perguntas sem resposta ou contendo entidades fora do contexto.

4.2 Resultados

Tabela 1. Resultados no conjunto de teste (14 perguntas, não usado no desenvolvimento).

Modo	Acurácia	Multi-hop	Precisão	Recall ctx.	Abstenção	Não sup.↓
RAG vetorial	14%	0%	17%	88%	0%	14%
GraphRAG híbrido	57%	14%	46%	100%	100%	0%
KAG (forma lógica)	**79%**	**57%**	**60%**	100%	100%	**0%**

Tabela 2. Resultados no conjunto de desenvolvimento (21 perguntas; otimista por ter guiado o desenvolvimento).

Modo	Acurácia	Multi-hop	Precisão	Recall ctx.	Abstenção	Não sup.↓
RAG vetorial	24%	18%	20%	97%	0%	14%
GraphRAG híbrido	71%	46%	56%	100%	100%	0%
KAG (forma lógica)	100%	100%	82%	100%	100%	0%

As Tabelas 1 e 2 mostram o mesmo padrão nos dois conjuntos. O resultado mais revelador é a distância entre recall de contexto e acurácia no RAG vetorial: em 88% das perguntas de teste a informação necessária estava nos *chunks* recuperados, mas a resposta só foi correta em 14%. Recuperar trechos semelhantes à pergunta não basta quando a resposta exige compor fatos de documentos diferentes. O GraphRAG eleva o recall de contexto a 100% e acerta todas as perguntas de um salto, mas, sem decompor a pergunta, ainda falha na maioria das multi-hop; o KAG, ao executar a cadeia de relações no grafo, chega a 57% nelas no teste.

Quanto à alucinação, os dois modos com grafo se abstiveram em todas as perguntas sem resposta — tanto as que pedem uma relação ausente (“Quem orientou o doutorado de Grace Hopper?”) quanto as com premissa falsa (“Em que ano Dennis Ritchie ganhou o Prêmio Nobel?”, “Quem criou a linguagem Python?”) — e não produziram nenhuma resposta não suportada. O baseline vetorial respondeu a todas com a frase mais similar do contexto. A latência média foi de 3 ms no modo vetorial e de 12–16 ms nos modos com grafo, no backend em memória.

4.3 Análise de erros

As três falhas do KAG no conjunto de teste e a baixa precisão de uma quarta pergunta indicam limitações concretas do planejador heurístico:

- **Ambiguidade lexical** (“Quem criou a linguagem que foi influenciada pela linguagem B?”): a pista “criou” mapeia tanto `CREATED` quanto `DEVELOPED_AT`; a partir de uma tecnologia, ambas são válidas, e o plano retornou o laboratório em vez da pessoa.
- **Salto implícito** (“Em que país nasceu Edsger Dijkstra?”): a pergunta pede um país, mas só o verbo “nasceu” aparece; falta o passo `LOCATED_IN`, e a resposta parou na cidade (Roterdã).
- **Atalho no caminho mínimo** (“Quem trabalhou no MIT e criou a linguagem Lisp?”): a aresta direta `Lisp-DEVELOPED_AT->MIT` torna o caminho mínimo de comprimento 1, sem nó intermediário do tipo Pessoa. Um plano de interseção (`MIT<-WORKED_AT-x`, `Lisp<-CREATED-x`) resolveria o caso.
- **Restrição por atributo** (“Quem recebeu o Prêmio Turing em 1974?”): a resposta correta foi incluída, mas junto com todos os laureados, pois o vocabulário de operadores não tem filtro por propriedade (precisão de 1/7).

Os quatro casos são cobertos pelo formato de plano usado pelo planejador LLM (interseção e tipos-alvo explícitos), e o terceiro e o quarto sugerem extensões diretas do vocabulário de operadores.

5 Discussão e Limitações

Os números indicam tendências, não significância estatística: o corpus e os conjuntos de perguntas são pequenos. O modo *offline* penaliza principalmente o baseline vetorial, cujo gerador extrativo não consegue compor fatos de duas frases; com um LLM, espera-se que a distância diminua nas perguntas de um salto, mas o recall de contexto e a ausência de raciocínio explícito continuam a favorecer os modos com grafo nas perguntas multi-hop. O extrator por regras depende do dicionário de entidades para reconhecer nomes novos (o cabeçalho `subject` do documento ameniza isso), e os *embeddings* por *hashing* capturam sobreposição lexical, não sinonímia. Por fim, a regra de abstenção por nome próprio desconhecido é conservadora: perguntas que citam eventos não modelados (por exemplo, “Segunda Guerra Mundial”) também levam à abstenção.

6 Conclusão

Apresentamos um sistema GraphRAG/KAG completo sobre Neo4j que cobre extração de entidades e relações com resolução de entidades, modelagem LPG com constraints e índices RANGE, full-text e vetoriais, recuperação híbrida com fusão RRF, travessia multi-hop enriquecida por PageRank e comunidades, raciocínio por forma lógica com Cypher seguro e uma interface que expõe o subgrafo, os fatos, o plano, as consultas e a confiança de cada resposta. No conjunto de teste, a decomposição lógica foi o fator decisivo para perguntas multi-hop, e os mecanismos de aterramento eliminaram as respostas não suportadas. Como trabalhos futuros, propomos operadores de filtro por atributo e de agregação, avaliação com LLMs locais e comerciais em conjuntos maiores e comunidades hierárquicas com Leiden para perguntas globais. O código, os dados e os scripts de avaliação estão no repositório do projeto.

Referências

Angles, R. (2018). The property graph database model. In *Proceedings of the 12th Alberto Mendelzon International Workshop on Foundations of Data Management (AMW)*, volume 2100 of *CEUR Workshop Proceedings*.

Blondel, V. D., Guillaume, J.-L., Lambiotte, R., and Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008.

Cormack, G. V., Clarke, C. L. A., and Buettcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., de Melo, G., Gutierrez, C., Kirrane, S., Gayo, J. E. L., Navigli, R., Neumaier, S., et al. (2021). Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.

Liang, L., Sun, M., Gui, Z., Zhu, Z., Jiang, Z., Zhong, L., Qu, Y., Zhao, P., Bo, Z., Yang, J., et al. (2024). KAG: Boosting LLMs in professional domains via knowledge augmented generation. *arXiv preprint arXiv:2409.13731*.

Malkov, Y. A. and Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836.

Page, L., Brin, S., Motwani, R., and Winograd, T. (1999). The PageRank citation ranking: Bringing order to the web. Technical report, Stanford InfoLab.

Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., and Wu, X. (2024). Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599.

Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., and Tang, S. (2024). Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921*.

Robinson, I., Webber, J., and Eifrem, E. (2015). *Graph Databases: New Opportunities for Connected Data*. O'Reilly Media, 2 edition.

Traag, V. A., Waltman, L., and van Eck, N. J. (2019). From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports*, 9:5233.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural*

Language Processing (EMNLP), pages 2369–2380.